

CODING CLUB

EPITECH

{EPITECH}
ACADEMY

Sujet : Défense d'Hyrule

I. Introduction

Vous avez franchi les portes du château, mais Ganon n'a pas dit son dernier mot. Pour protéger Hyrule, la princesse Zelda doit réactiver les anciens sceaux Sheikah — et cette fois, ils sont **brisés**.

Chaque sceau est une grille de neuf runes sur neuf dont certaines cases ont été effacées par la magie de Ganon. Vous seul pouvez les restaurer. La règle sacrée reste la même : sur chaque **ligne**, chaque **colonne** et chaque **bloc** de trois par trois, les neuf runes (de 1 à 9) doivent apparaître une seule fois.

Vous n'êtes plus un simple vérificateur : vous devez **résoudre** le sceau. Reconstituez la grille complète, et la lumière de la Triforce repoussera les ténèbres.

Le but du projet est de coder, en Python, un résolveur de sudoku.

II. Les runes anciennes

Comme lors de l'Assaut, le terminal Sheikah met trois fonctions à votre disposition. Servez-vous-en !

fonction	ce qu'elle vous rend
<code>getLine(ligne: list) -> list</code>	la liste des runes déjà présentes sur une ligne
<code>getColumn(sudoku: list, index: int) -> list</code>	la liste des runes déjà présentes dans une colonne
<code>getBlock(sudoku: list, x: int, y; int) -> list</code>	la liste des runes déjà présentes dans le bloc contenant la case (x, y)

III. Restaurer le sceau

Le terminal Sheikah vous fournit trois sceaux brisés, tous **toujours résolubles**. Chargez-en un avec l'une de ces fonctions, qui vous rend une grille de neuf listes de neuf entiers (les cases effacées valent 0) :

fonction	sceau
<code>charger_facile()</code>	le plus doux pour commencer
<code>charger_moyen()</code>	un cran au-dessus
<code>charger_difficile()</code>	le sceau ultime, celui qui sauve Hyrule

La force brute est votre alliée. Voici l'idée maîtresse, celle que murmurent les Sheikah :

Pour chaque case vide, notez toutes les runes qui pourraient encore s'y trouver (de 1 à 9), puis **retirez peu à peu** celles qui sont déjà présentes sur la même ligne, la même colonne ou le même bloc. Lorsqu'il ne reste qu'une seule possibilité, cette rune est la bonne.

Concrètement, remplacez chaque 0 par la liste [1, 2, 3, 4, 5, 6, 7, 8, 9], puis élaguez cette liste tour après tour jusqu'à ce que chaque case ne contienne plus qu'un unique entier. Répétez tant que la grille n'est pas entièrement remplie.

Voici une piste d'implémentation :

```
grille = charger_difficile()

# Chaque case vide devient la liste de toutes les runes encore possibles
sudoku = [[v if v != 0 else [i for i in range(1, 10)] for v in ligne] for ligne in
grille]

# Tant que des cases contiennent encore une liste de possibilités, retirez les
runes déjà présentes sur la ligne, la colonne et le bloc, puis fixez les cases où
il ne reste qu'une seule possibilité.
# ... et appelez afficher(sudoku) à la fin de chaque tour.
```

****Appelez `afficher(sudoku)` à chaque tour pour voir le sceau se restaurer sous vos yeux — c'est instructif, et le terminal compte le nombre de tours. Le sceau s'illumine en vert**** lorsqu'il est complet et cohérent, et rougeoie là où une règle est violée (la première ligne, colonne ou bloc contenant une rune en double).

****Le royaume n'est sauvé que lorsque le sceau `charger_difficile()` est entièrement restauré.****

IV. La légende du héros

Vous avez repoussé Ganon. Hyrule vous doit sa survie !

Bonus : repoussez encore plus loin les limites de votre résolveur. Certains sceaux résistent à la simple élimination et exigent un peu de déduction supplémentaire — saurez-vous les percer ? Si vous réussissez vous obtiendrez la 'golden mark'

Si vous êtes curieux, pensez à poser vos questions aux Mantas. Ils connaissent bien des secrets d'Hyrule.